

Containerizing a MERN Stack Application using Docker and Docker Compose

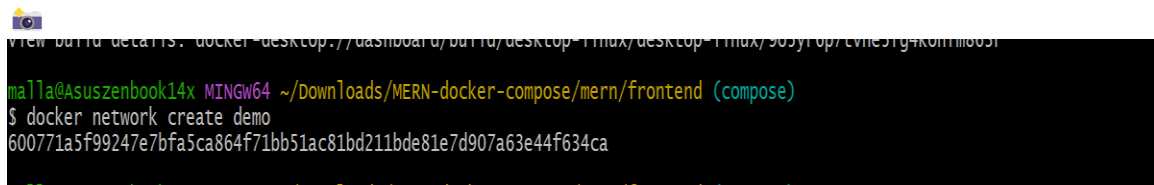
In this project, I containerized a simple MERN (MongoDB, Express, React, Node.js) stack application using Docker. The goal was to understand how each part of a full-stack application can be containerized and connected through a Docker network. Initially, I worked with individual Dockerfiles for the frontend, backend, and MongoDB, and finally automated the process using Docker Compose.

1. Creating a Docker Network

To allow communication between containers, I created a custom network named 'demo'.

Command:

```
docker network create demo
```



```
ma11a@Asuszenbook14x MINGW64 ~/Downloads/MERN-docker-compose/mern/frontend (compose)
$ docker network create demo
600771a5f99247e7bfa5ca864f71bb51ac81bd211bde81e7d907a63e44f634ca
```

2. Containerizing the Frontend (React)

Navigated to the frontend directory and wrote a Dockerfile to build the React app image.

Commands:

```
cd mern/frontend
```

```
docker build -t mern-frontend .
```

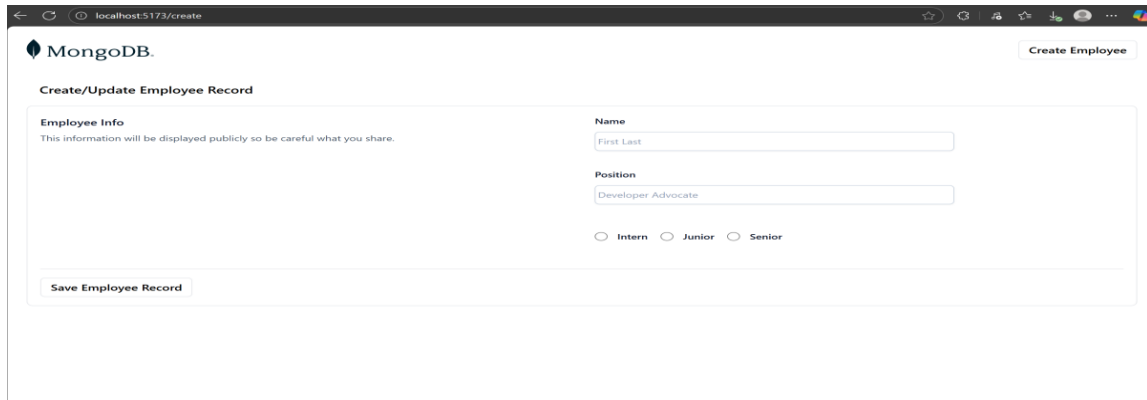
Then, ran the frontend container:

```
docker run --name=frontend --network=demo -d -p 5173:5173 mern-frontend
```

Verified the frontend by visiting <http://localhost:5173>.



```
ma11a@Asuszenbook14x MINGW64 ~/Downloads/MERN-docker-compose/mern/frontend (compose)
$ docker run --name=frontend --network=demo -d -p 5173:5173 mern-frontend
a5d1a285165cc179e9da3c4f37395ba81d86d509e94ab69a78a890def217c90c
```



3. Running MongoDB Container

Pulled the official MongoDB image and ran it in the same network.

Command:

```
docker run --network=demo --name mongodb -d -p 27017:27017 -v ~/opt/data:/data/db mongo:latest
```

```
ma11a@Asuszenbook14x MINGW64 ~/Downloads/MERN-docker-compose/mern/frontend (compose)
$ docker run --network=demo --name mongodb -d -p 27017:27017 -v ~/opt/data:/data/db mongo:latest
Unable to find image 'mongo:latest' locally
latest: Pulling from library/mongo
4b3ffd8ccb52: Pull complete
34baad995762: Pull complete
5d6f7cf8e15d: Pull complete
f90aba9940a7: Pull complete
5d3652024acd: Pull complete
74522aa03d00: Pull complete
ce8dccf1f4c1: Pull complete
a4ada45c862e: Pull complete
Digest: sha256:7d1a1a613b41523172dc2b1b02c706bc56cee64144ccd6205b1b38703c85bf61
Status: Downloaded newer image for mongo:latest
d623ba27f4ee6ed1eb125b9884fb8713e29878fef2cbfd6c911f42a89db0c35d
```

4. Containerizing the Backend (Express + Node.js)

Navigated to the backend directory and created a Dockerfile to containerize the Node.js server.

Commands:

```
cd mern/backend
docker build -t mern-backend .
```

Then ran the backend container:

```
docker run --name=backend --network=demo -d -p 5050:5050 mern-backend
```

```
$ docker build -t mern-backend .
[+] Building 8.6s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 168B
=> [internal] load metadata for docker.io/library/node:18.9.1
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/node:18.9.1@sha256:d6ed353d022f6313aa7c3f3df69f3a216f1c9f8c3374502eb5e6c45088ce68e8
=> [internal] load build context
=> transferring context: 35.98kB
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY package.json .
=> [4/5] RUN npm install
=> [5/5] COPY . .
=> exporting to image
=> exporting layers
=> writing image sha256:abacfa9311e299866ec413714cff9f3a7e6ee36963a3e22455c771c0ecc748be
=> naming to docker.io/library/mern-backend
```

```
ma11a@Asuszenbook14x MINGW64 ~/Downloads/MERN-docker-compose/mern/backend (compose)
$ docker run --name=backend --network=demo -d -p 5050:5050 mern-backend
cfff212030018d6aeef89c9ea06ccf6a6817c0756e321e8cc2d42bd34653812da
```

5. Testing Containers

Verified that all containers (frontend, backend, and MongoDB) were running properly and accessible via their respective ports (5173, 5050, 27017).



Create Employee

Create/Update Employee Record

Employee Info
This information will be displayed publicly so be careful what you share.

Name
First Last

Position
Developer Advocate

☐ Intern ☐ Junior ☐ Senior

Save Employee Record



Create Employee

Employee Records

Name	Position	Level	Action
HARI NATH REDDY MALLANGI	devops engineer	Intern	EditDelete

6. Simplifying with Docker Compose

Wrote a docker-compose.yml file to automate building and running all containers simultaneously.

Command:

docker compose up -d

With this single command, all services (frontend, backend, and MongoDB) started automatically, connected through the same network.

 removing the containers created individually

```
ma11a@Asuszenbook14x MINGW64 ~/Downloads/MERN-docker-compose/mern/backend (compose)
$ docker rm -f cff212030018 d623ba27f4ee a5d1a285165c
cff212030018
d623ba27f4ee
a5d1a285165c
```

🚀 After running docker compose up -d

```
mern@msbssimons-mbp:~/Downloads/mern-docker-compose/mern-backend (compose)$ docker compose up -d
[+] Building 4.5s (22/21) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 8088                                           0.0s
=> [frontend internal] load build definition from Dockerfile            0.0s
=> => transferring dockerfile: 687B                                     0.0s
=> [backend internal] load build definition from Dockerfile            0.1s
=> => transferring dockerfile: 168B                                     0.0s
=> [frontend internal] load metadata for docker.io/library/node:18.9.1 1.7s
=> [auth] library/node:pull token for registry-1.docker.io             0.0s
=> [backend internal] load .dockerignore                                0.1s
=> => transferring context: 2B                                           0.0s
=> [frontend internal] load .dockerignore                                0.0s
=> => transferring context: 1.17kB                                       0.0s
=> [backend 1/5] FROM docker.io/library/node:18.9.1sha256:d6ed35d022f613aa7c3f3df69f3a216f1c9f8c3374502eb5e6c45088ce68e8 0.0s
=> [frontend internal] load build context                               0.1s
=> => transferring context: 1.17kB                                       0.0s
=> [backend internal] load build context                               0.1s
=> => transferring context: 251B                                         0.0s
=> CACHED [backend 2/5] WORKDIR /app                                   0.0s
=> CACHED [frontend 3/5] COPY package.json .                           0.0s
=> CACHED [frontend 4/5] RUN npm install                               0.0s
=> CACHED [frontend 5/5] COPY . .                                       0.0s
=> CACHED [backend 3/5] COPY package.json .                           0.0s
=> CACHED [backend 4/5] RUN npm install                               0.0s
=> CACHED [backend 5/5] COPY . .                                       0.0s
=> [frontend] exporting to image                                       0.1s
=> => exporting layers                                                  0.0s
=> => writing image sha256:fd9342d864474c40bf71f4fbd2571d90cdcdccld702a1c68995a02e0e2762b3 0.0s
=> => naming to docker.io/library/mern-docker-compose-frontend        0.0s
=> [backend] exporting to image                                       0.0s
=> => exporting layers                                                  0.0s
=> => writing image sha256:abacfa9331e299866ec413714cff9f3e7e6ee36963a3e22455c771c0ecc749be 0.0s
=> => naming to docker.io/library/mern-docker-compose-backend          0.0s
=> [backend] resolving provenance for metadata file                   0.0s
=> [frontend] resolving provenance for metadata file                   0.0s
[+] Running 7/7
✔ frontend                               Built                                0.0s
✔ backend                               Built                                0.0s
✔ Network mern-docker-compose_mern_network Created                       0.1s
✔ Volume "mern-docker-compose_mongo-data" Created                       0.0s
✔ Container mern-docker-compose-mongodb-1 Started                       0.8s
✔ Container mern-docker-compose-frontend-1 Started                      0.7s
✔ Container mern-docker-compose-backend-1 Started                       1.0s
```

7. Conclusion

This exercise helped me understand how to efficiently containerize multi-service applications. Docker Compose greatly simplifies the management of multiple interconnected containers and is ideal for microservice-based architectures like the MERN stack.

Through this project, I gained hands-on experience with Docker networking, container orchestration, and Compose-based automation. This practical understanding strengthens my DevOps foundation, especially in deploying and managing multi-container applications efficiently.